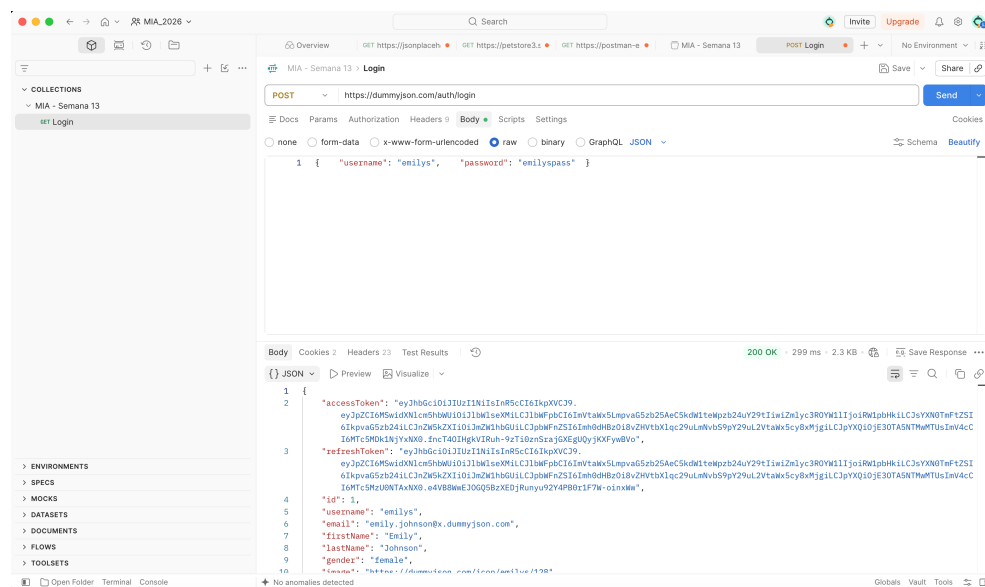
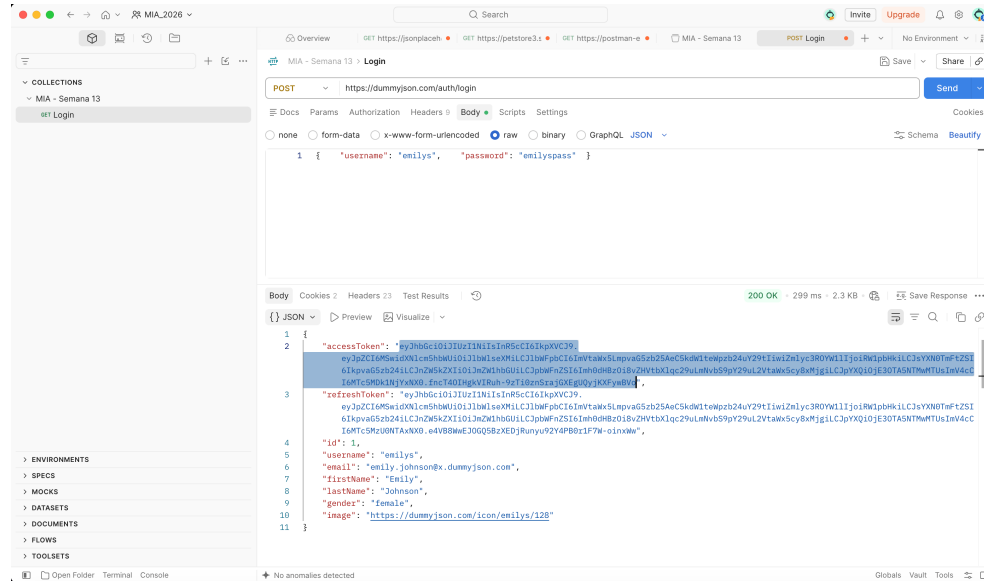
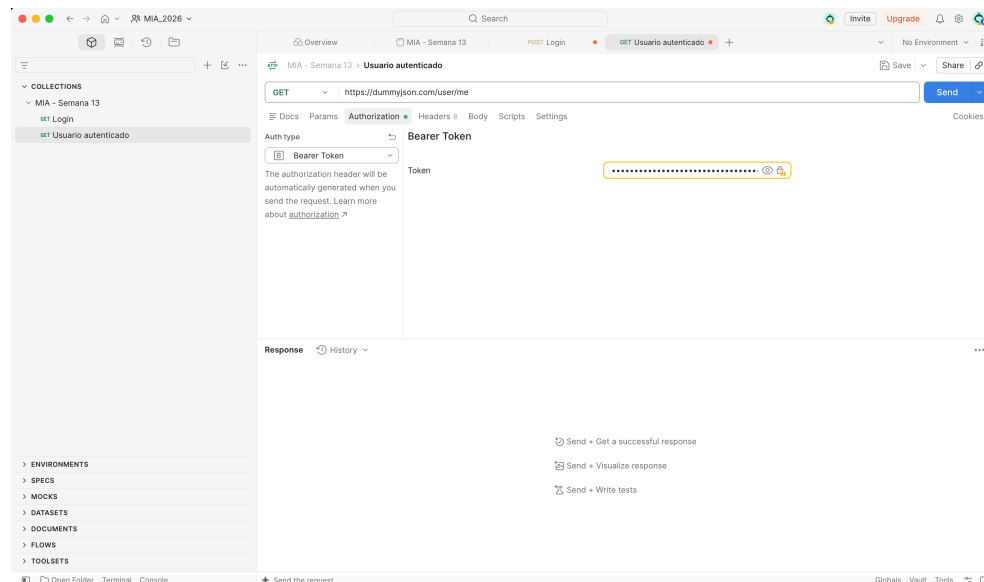


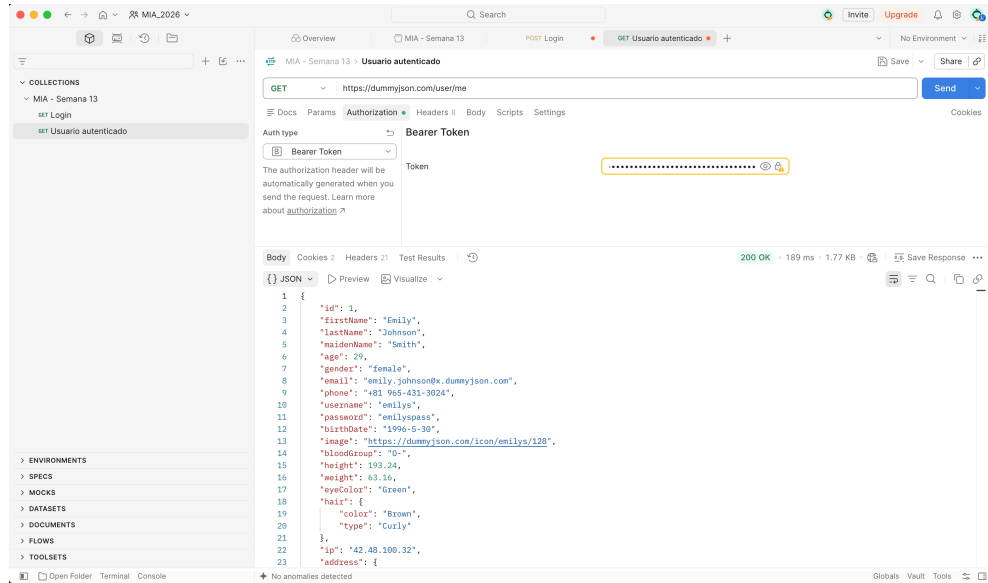




2. Utilizar token

- Cree una segunda solicitud llamada: Usuario autenticado.
- Seleccione GET.
- Utilice: `https://dummyjson.com/user/me`
- En Authorization seleccione Bearer Token.
- Copie el accessToken obtenido en la solicitud Login.
- Ejecute la solicitud.
- Verifique que la API devuelve la información del usuario autenticado.

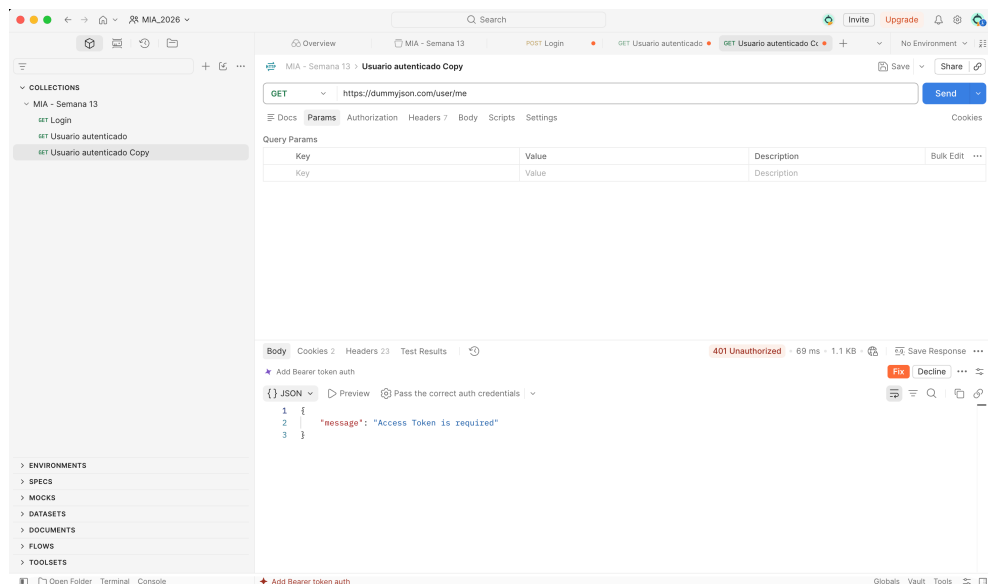


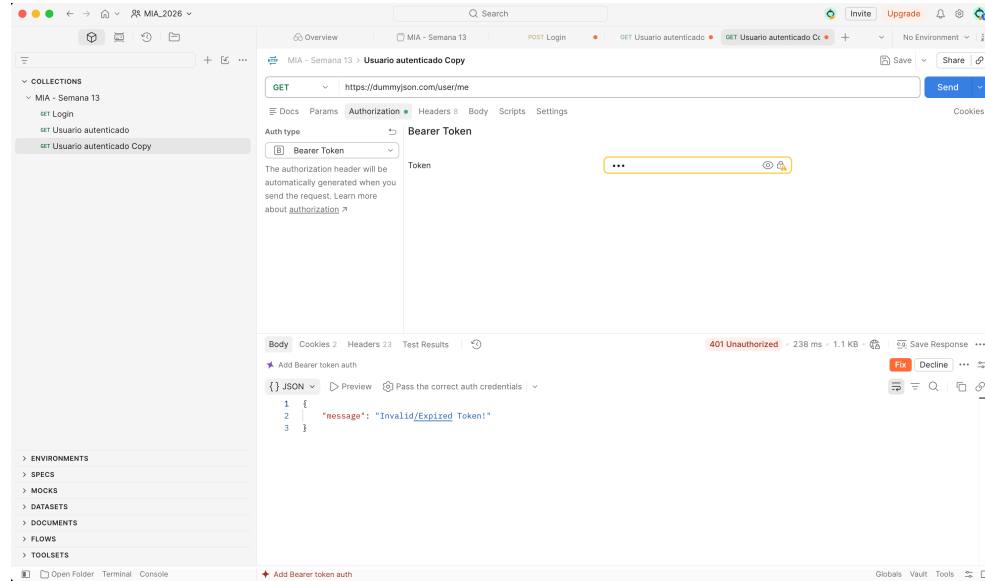


Resultado: La solicitud autenticada se ejecutó correctamente y la API devolvió la información correspondiente al usuario autenticado.

3. Comprobar qué ocurre sin autenticación

- Duplique la solicitud Usuario autenticado.
- Quite el token.
- Ejecute nuevamente la solicitud.
- Compare la respuesta con la solicitud autenticada.
- Registre el código HTTP y explique la diferencia.





Resultado: Al ejecutar la solicitud sin proporcionar el Bearer Token, la API respondió con el código HTTP 401 Unauthorized, indicando que se requiere un token de acceso para consultar el recurso. En cambio, al realizar la solicitud con el Bearer Token válido, la API respondió con 200 OK y devolvió la información del usuario autenticado. La diferencia se debe a que el endpoint /user/me requiere autenticación para permitir el acceso.

4. Evidencia

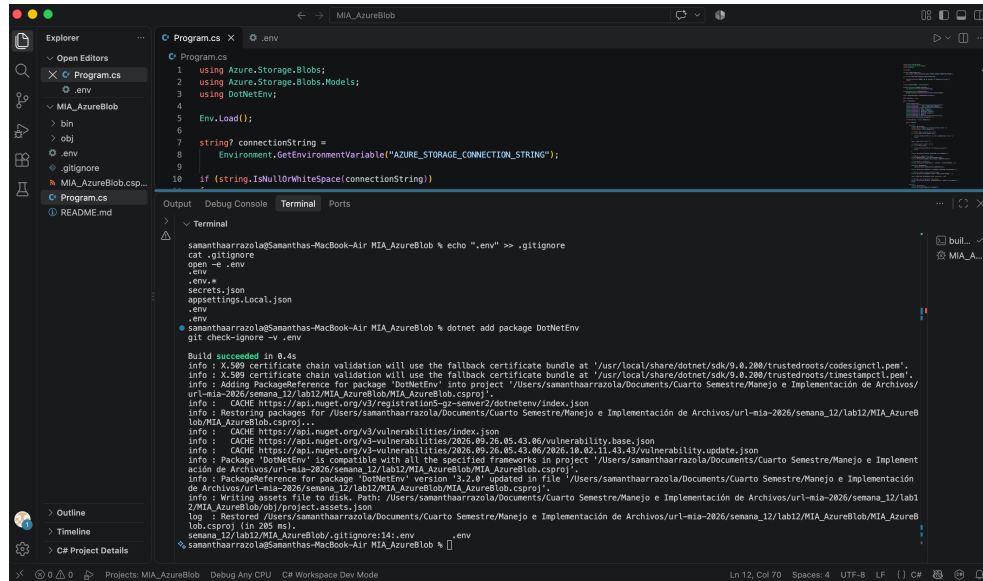
- Captura de Login mostrando la respuesta.
- Captura de la solicitud autenticada.
- Captura de la prueba sin token.
- Indique qué función cumple el Bearer Token.

La documentación de DummyJSON muestra el flujo de login, la obtención del accessToken y su envío mediante el encabezado Authorization: Bearer.

Función del Bearer Token

El Bearer Token funciona como una credencial de autenticación que permite verificar que el usuario está autorizado para acceder a un recurso protegido. El token obtenido durante el inicio de sesión se envía en el encabezado Authorization de las solicitudes posteriores. Si el token es válido, la API permite el acceso al recurso; si no se proporciona o no es válido, la solicitud es rechazada.

La idea es que el programa siga utilizando la misma variable para conectarse a Azure, pero ahora el valor será obtenido desde el entorno y no desde el código



```

1 using Azure.Storage.Blobs;
2 using Azure.Storage.Blobs.Models;
3 using DotNetEnv;
4
5 Env.Load();
6
7 string? connectionString =
8     Environment.GetEnvironmentVariable("AZURE_STORAGE_CONNECTION_STRING");
9
10 if (string.IsNullOrEmpty(connectionString))

```

```

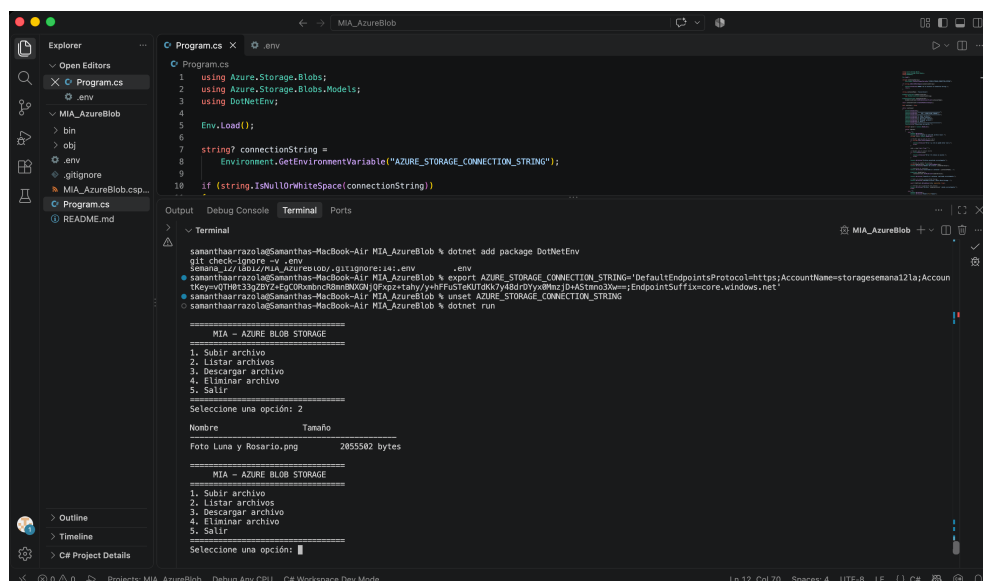
samantharrazola@Samanthas-MacBook-Air MIA_AzureBlob % echo ".env" >> .gitignore
cat .gitignore
.gitignore
.env
.env
secrets.json
appsettings.Local.json
.env
samantharrazola@Samanthas-MacBook-Air MIA_AzureBlob % dotnet add package DotNetEnv
git check-ignore -v .env
Build succeeded in 0.4s
info : X.509 certificate chain validation will use the fallback certificate bundle at '/usr/local/share/dotnet/sdk/9.0.200/trustedroots/codesignctl.pem'.
info : X.509 certificate chain validation will use the fallback certificate bundle at '/usr/local/share/dotnet/sdk/9.0.200/trustedroots/timesignctl.pem'.
info : Adding PackageReference for package 'DotNetEnv' into project '/Users/samantharrazola/Documents/Cuarto Semestre/Manejo e Implementación de Archivos/ur-mia-2026/semana_12/lab12/MIA_AzureBlob/MIA_AzureBlob.csproj'.
info : CACHE https://api.nuget.org/v3/registrations-global/latest/dotnetenv/index.json
info : Restoring packages for /Users/samantharrazola/Documents/Cuarto Semestre/Manejo e Implementación de Archivos/ur-mia-2026/semana_12/lab12/MIA_AzureBlob/MIA_AzureBlob.csproj...
info : CACHE https://api.nuget.org/v3/vulnerabilities/2026.09.26.05.43.86/vulnerability.base.json
info : CACHE https://api.nuget.org/v3/vulnerabilities/2026.09.26.05.43.86/2026.10.02.11.43.43/vulnerability.update.json
info : Package 'DotNetEnv' is compatible with all the specified frameworks in project '/Users/samantharrazola/Documents/Cuarto Semestre/Manejo e Implementación de Archivos/ur-mia-2026/semana_12/lab12/MIA_AzureBlob/MIA_AzureBlob.csproj'.
info : PackageReference for package 'DotNetEnv' version '3.2.0' updated in file '/Users/samantharrazola/Documents/Cuarto Semestre/Manejo e Implementación de Archivos/ur-mia-2026/semana_12/lab12/MIA_AzureBlob/MIA_AzureBlob.csproj'.
info : Writing assets file to disk. Path: /Users/samantharrazola/Documents/Cuarto Semestre/Manejo e Implementación de Archivos/ur-mia-2026/semana_12/lab12/MIA_AzureBlob/obj/project.assets.json
log : Restored /Users/samantharrazola/Documents/Cuarto Semestre/Manejo e Implementación de Archivos/ur-mia-2026/semana_12/lab12/MIA_AzureBlob/MIA_AzureBlob.csproj (in 285 ms).
samantharrazola@Samanthas-MacBook-Air MIA_AzureBlob % dotnet run
samantharrazola@Samanthas-MacBook-Air MIA_AzureBlob %

```

Resultado: La Connection String fue eliminada del código fuente y ahora se obtiene mediante la variable de entorno AZURE_STORAGE_CONNECTION_STRING, evitando almacenar directamente la credencial dentro del programa.

7. Crear la variable para ejecutar localmente

- En la carpeta principal del proyecto, cree un archivo llamado .env.
- Dentro del archivo agregue la variable:
AZURE_STORAGE_CONNECTION_STRING=TU_CONNECTION_STRING
- Reemplace SU_CONNECTION_STRING por la Connection String de su Storage Account.
- Verifique que el programa lea la variable desde el .env y ejecute nuevamente el programa.
- Compruebe que el programa continúa funcionando correctamente.
- No publique el archivo .env en GitHub**, ya que contiene la credencial real.



```

1 using Azure.Storage.Blobs;
2 using Azure.Storage.Blobs.Models;
3 using DotNetEnv;
4
5 Env.Load();
6
7 string? connectionString =
8     Environment.GetEnvironmentVariable("AZURE_STORAGE_CONNECTION_STRING");
9
10 if (string.IsNullOrEmpty(connectionString))

```

```

samantharrazola@Samanthas-MacBook-Air MIA_AzureBlob % dotnet add package DotNetEnv
git check-ignore -v .env
samantharrazola@Samanthas-MacBook-Air MIA_AzureBlob % dotnet run
samantharrazola@Samanthas-MacBook-Air MIA_AzureBlob % export AZURE_STORAGE_CONNECTION_STRING='DefaultEndpointProtocol=https;AccountName=storageesemana12la;AccountKey=QTh8T3q2Bz74qQrmbnQ8m8Q0NjQFpxzTahyYvHFFJSTeUtk7Y48rDyXWmzjd4Stmo3w==;EndpointSuffix=core.windows.net'
samantharrazola@Samanthas-MacBook-Air MIA_AzureBlob % dotnet run
samantharrazola@Samanthas-MacBook-Air MIA_AzureBlob %

```

```

MIA - AZURE BLOB STORAGE
1. Subir archivo
2. Listar archivos
3. Descargar archivo
4. Eliminar archivo
5. Salir
Seleccione una opción: 2
Nombre      Tamaño
Foto Luna y Rosario.png  2655582 bytes
MIA - AZURE BLOB STORAGE
1. Subir archivo
2. Listar archivos
3. Descargar archivo
4. Eliminar archivo
5. Salir
Seleccione una opción:

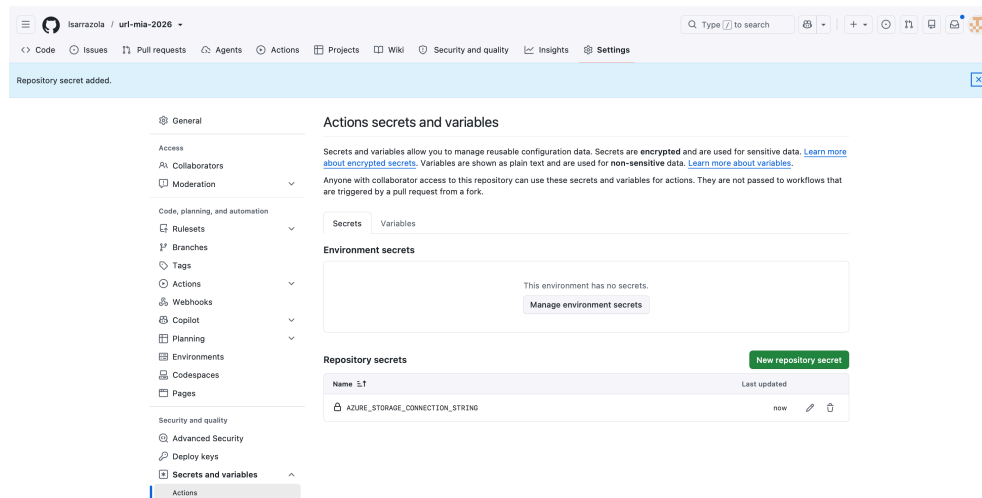
```

Resultado: Se configuró el archivo .env con la variable AZURE_STORAGE_CONNECTION_STRING y se ejecutó nuevamente la aplicación. Se comprobó que el programa continúa funcionando correctamente y puede conectarse a Azure Storage sin almacenar la Connection String directamente en el código fuente.



8. Crear el Secret en Github

- Ingrese al repositorio de su proyecto en GitHub.
- Seleccione Settings.
- En el menú lateral seleccione Secrets and variables.
- Seleccione Actions.
- Seleccione New repository secret.
- En Name escriba: AZURE_STORAGE_CONNECTION_STRING
- En Secret coloque la Connection String de Azure Storage.
- Seleccione Add secret.



Resultado: Se creó correctamente el Repository Secret AZURE_STORAGE_CONNECTION_STRING en GitHub Actions, permitiendo almacenar la credencial de Azure Storage sin exponer su valor dentro del repositorio.

Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab2/ y coloque en ella un PDF con la solución del laboratorio. Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.